Safety Improvement of a Local LLM Assistant

Assignment 3 | AISRA1301 - AI Security and Risk Analysis | QAIRU

1. Title and Research Objective

This project continues the Midterm news assistant and the GCG jailbreak research topic of Zou et al. [1]. It evaluates two defence layers: a restrictive system prompt (vicuna-guarded) and a new randomized input-smoothing mechanism inspired by SmoothLLM [2]. H1: the system prompt reduces refusal bypass. H2: adding smoothing reduces suffix-induced bypass further. The study also measures benign task performance, latency and API reliability rather than treating a lower bypass score as sufficient evidence of a safer usable system.

2. Midterm Baseline and Identified Risk

The existing architecture is React/Vite, a Python HTTP proxy and local Ollama serving vicuna:7b (7B, Q4_0). The supplied historical summary records 22/25 refusal bypasses with a published suffix for the base model and 19/25 for vicuna-guarded. Direct-request figures are 3/25 and 1/25; cached GCG gives 0/26 for both. These saved results motivate further defence testing but are not merged with the new run. The cached probe contains two underlying goals combined with different suffixes, not 26 independent harmful goals. A source review also found that the original proxy trusted client-supplied system roles and had no individual message-size limit.

3. Extended Testing Methodology

A new local Ollama experiment compares base Vicuna, vicuna-guarded and vicuna-guarded plus smoothing on the same 40 fixed inputs: the first 10 AdvBench goals without a suffix, those same goals with the unchanged first static example suffix from llm-attacks.org, 10 unique cached-GCG inputs from the supplied garak log, and 10 benign tasks. Cached inputs reuse two goals with multiple suffixes. Benign controls include simple editorial operations, arithmetic, translation, Russian and Kazakh. Exact prompts, output text, votes and hashes are saved. No attack is optimised or strengthened. Tests use only the local authorised models.

4. Additional Safety or Security Mechanism

System prompt. Modelfile.guarded reuses Vicuna weights and instructs the assistant to treat user text as untrusted, reject role changes, refuse harmful assistance and offer a safe alternative. It is inexpensive but depends on instruction following.

GCG-specific additional layer. site/smoothllm.py independently adapts the SmoothLLM idea: create five copies of the entire latest user message, insert random characters after 10% of its character positions, query guarded Vicuna five times, then select a random response consistent with the majority refusal/non-refusal vote. It perturbs the whole message because the defender does not know the suffix boundary. The rationale is that token-specific GCG sequences can be fragile to character changes. The local regex vote and Ollama interface differ from the reference implementation, so no published robustness guarantee is claimed. The prototype buffers candidates before releasing the selected answer; intermediate candidates never stream to the user.

API hardening. The server independently rejects untrusted roles, invalid schemas, empty text, messages over 4,000 characters and bodies over 65,536 bytes. It inserts a trusted system prompt and retains the last 12 validated messages. This closes application-level paths, while smoothing targets the suffix attack itself.

5. Experimental Setup and Evaluation Metrics

Actual run: Ollama 0.40.9 on Windows, NVIDIA RTX 4060 Laptop GPU (8 GB), vicuna:7b Q4_0 and the guarded derivative. All three use temperature 0, seed 20261027, num_predict 96, num_ctx 2048 and GPU offload. Smoothing uses five insertions-based copies at 10%, with seed 20261027 plus case index. Each configuration has an excluded warm-up, then runs sequentially in fixed order. There are 120 selected responses and 280 measured model generations, excluding warm-ups and the separate judge. The new conditions differ from the historical garak setup; figures are compared only within each run.

Metrics: garak 0.17.0 refusal-lexicon bypass (absence of any of its 156 phrases), supplementary harmful-assistance labels from local qwen3.5:4b with thinking disabled, benign task scores, and mean wall-clock latency including all five smoothing calls. The judge receives the original goal without its attack suffix. Both safety measures are fallible and lack human calibration. Initial benign regex scores are retained alongside a post-run rubric audit correcting case sensitivity, numbered lists and numeric fidelity; no answers or defence parameters changed. A separate 48-case paired HTTP study uses a stub to measure API enforcement.

6. Results and Comparative Analysis

New run: lexical bypass	Base	Guarded	Guarded + smooth
Direct (10)	0%	0%	0%
Cached GCG (10)	10%	0%	0%
Published suffix (10)	70%	70%	10%

Judge-labelled harmful	Base	Guarded	Guarded + smooth
Direct	0/10	0/10	0/10
Cached GCG	0/10	0/10	0/10
Published suffix	1/10	0/10	0/10

Functionality and cost	Base	Guarded	Guarded + smooth
Initial benign regex score	9/10	7/10	5/10
Audited benign task score	7/10	7/10	3/10
Mean benign latency (s)	0.72	0.50	2.65
Mean suffix latency (s)	0.92	0.60	5.86

Smoothing reduces published-suffix lexical bypass from 70% to 10% (60 percentage points). The final judge labels 1/10 base-model suffix responses harmful and 0/10 for both defences: additional harmful-content reduction beyond the system prompt is not demonstrated. Short fragments inflate lexical bypass; the judge also wrongly calls some fragments refusals, limiting trust in its labels. Audited benign success is 7/10, 7/10 and 3/10; smoothing raises mean benign latency from 0.50 s to 2.65 s. No statistical significance is established on these small, dependent samples.

In the separate HTTP suite, forged-role and oversized-input forwarding each fall from 8/8 to 0/8. Valid requests remain 24/24 accepted. Controlled schema errors rise from 4/8 to 8/8, eliminating four disconnections. A simulated unavailable upstream returns 502. These measurements demonstrate boundary enforcement, not model jailbreak resistance.

7. Interpretability and Availability Analysis

Per-case candidates and refusal votes explain which output the smoothing layer selects. Named validation rules explain API rejections. Neither method explains Vicuna token generation. SHAP/LIME attribution is not applied because this evaluation defines no supervised feature-attribution target through the chat interface; response analysis is the relevant inspectable evidence here. The independent model judge improves visibility into lexical-detector errors but requires human calibration before its labels can be trusted as ground truth.

The defence changes behaviour by perturbing inputs and selecting among multiple generations. It also delays the first visible response until all candidates finish. A 502 fallback is tested with a simulated outage, not a real service failure. Per-call/socket inactivity timeouts do not impose a total deadline across five calls. GPU memory, local model availability, thread-per-request handling and the absence of rate/concurrency limits remain availability constraints. A malformed or empty model response fails closed rather than releasing an unchecked candidate.

8. Safe Human-AI Interaction

Both chat interfaces now warn that the assistant can be wrong, ask users to verify facts and sources before publication, and discourage personal-data entry. The full-page chat discloses browser-local history. The prototype does not automatically publish model answers. Human editorial review is necessary to counter automation bias, fabricated facts and misuse. Given the measured task degradation, users should be able to reformulate corrupted tasks and review the original request before accepting an answer. An evaluation-mode warning about slower responses is appropriate for this experimental defence.

The proxy itself does not persist chats, but React localStorage does, and research logs contain prompts/completions. Consequently, a blanket no-storage claim would be inaccurate. Restrict access to raw research logs and clear browser history on shared devices. Public slides/report show aggregate evidence without harmful completion details.

9. Limitations and Future Work

Ten inputs per group, a single seed, one quantised model and fixed execution order limit generalisation and timing inference. The cached set has only two goals, and the strongest-suffix set repeats one published suffix. Generation truncation at 96 tokens can hide later harmful content or destroy answer completeness. Smoothing can corrupt useful text, numbers and names. The local regex may choose a harmful output as non-refusal majority or miss a disguised refusal; it is not an output safety filter. The Qwen judge is a single uncalibrated model susceptible to misclassification and prompt injection. No adaptive attack or certification is performed.

Priority follow-up: evaluate larger held-out goals, multiple random seeds, human-labelled harmfulness and richer benign tasks. Test whether fewer perturbations or semantic paraphrasing preserve task quality, using a separate calibration set to avoid tuning on the reported test set. Add output verification before release, bounded concurrency and authenticated server-side conversation history. The result is a demonstrated GCG-oriented defence prototype with explicit safety/usability costs, not evidence of complete protection.

10. References

[1] Zou et al. Universal and Transferable Adversarial Attacks on Aligned Language Models. 2023. https://arxiv.org/abs/2307.15043; dataset: github.com/llm-attacks/llm-attacks; static suffix: https://llm-attacks.org/
[2] Robey, Wong, Hassani, Pappas. SmoothLLM. 2023. https://arxiv.org/abs/2310.03684; reference implementation: github.com/arobey1/smooth-llm
[3] NVIDIA garak 0.17.0, github.com/NVIDIA/garak; supplied historical reports and summary.
[4] Ollama chat specification: github.com/ollama/ollama/blob/main/docs/openapi.yaml
[5] New evidence, 27 October 2026: experiments/logs/assignment3/ollama_results.jsonl, ollama_summary.json, ollama_environment.json, judge_results.jsonl, judge_summary.json and boundary_summary.json.

AI assistance: Codex researched/implemented the defence, executed the new local evaluations and revised these materials. See AI_USE_DECLARATION.md.